Count inversions

arr = [6, 3, 5, 2, 7]

6	3	5	2	7

inversions pairs
(arr[i], arr[j])

1. i < j
2. arr[i] > arr[j]

(6,3), (6,5), (6,2)
(3,2), (5,2) = 5

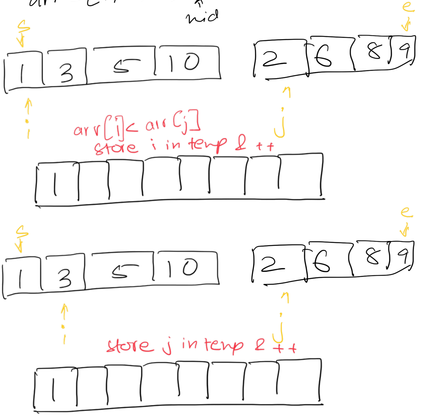
1. Brute force

```
for (i=0 to n) {
   for (j=i+1; j<n; j++) {
      if (arr[i] > arr[j]) {
         InvCount++
      }
   }
}
```

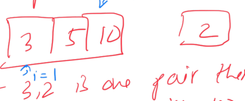
TC: $O(n^2)$

2. Optimal approach (merge sort)

arr = [1, 3, 5, 10, 2, 6, 8, 9]



BUT since 3 > 2 we got one inversion pair so if arr[i] > arr[j] we can increment invcount BUT at the same time we can try getting more pairs



if 3,2 is one pair then baki ke values after 3 will also result in a pair

mid - i + 1 = 3 - 1 + 1 = 3

3 total pairs

Code

```
int merge(arr, st, mid, end) {
   i = st, j = mid+1
   invcount = 0
   while(i <= mid && j <= end) {
      if (arr[i] <= arr[j]) {
         temp.pb(arr[i])
         i++
      } else {
         temp.pb(arr[j])
         j++;
         invCount += mid - i + 1
      }
   }
   rest same .....
   return invcount;
}

int mergeSort(arr, st, end) {
   if (st < end)
      mid = st + (end - st)/2
      mS(arr, st, mid)
      mS(arr, mid+1, end)
      merge(arr, st, mid, end)
      return LC + RC + invCount
   }
   return 0;
}
```

each have their own individual invcounts